

Deep Learning Engineering Report: Fine-tuning VISTA-3D for Small Nodule Segmentation

Mihai Bogdan Deaconu

December 24, 2025

Contents

1	Project Overview and Architectural Decisions	3
1.1	Context and Objective	3
1.2	Model Selection: Why VISTA-3D?	3
1.3	Engineering Constraints and Initial Hurdles	4
2	Data Engineering Pipeline	5
2.1	Phase 1: The Patch-Extraction Approach (Deprecated)	5
2.2	Phase 2: Full Volumetric Preprocessing (Current Standard)	5
2.2.1	Handling Legacy Dependencies (PyLIDC Patch)	6
2.2.2	Consensus and Geometry	6
2.3	The Spacing Hypothesis: 1.5mm vs. 0.7mm	6
3	Training Infrastructure and Optimizations	7
3.1	The Batch Size Constraint	7
3.2	Gradient Accumulation Implementation	7
3.3	Validation Latency and Inference Optimization	8
3.4	Context Window vs. Resolution	8
3.5	Loss Function Dynamics	8
4	Experimental Log and Results Analysis	9
4.1	Zero-Shot Baseline Evaluation	9
4.2	Phase 1: The Baseline (Dice 0.44 - 0.54)	9

4.3	Phase 2: The High-Resolution Era (Dice 0.35 - 0.56)	10
4.4	Analysis of "Model Collapse"	10
4.5	Conclusion on Metrics	10

Chapter 1

Project Overview and Architectural Decisions

1.1 Context and Objective

The primary objective of this project was to develop a high-precision 3D segmentation pipeline capable of identifying and delineating pulmonary nodules from Volumetric Computed Tomography (CT) scans. This task is critical for the "Virtual Biopsy" initiative, where accurate segmentation serves as the prerequisite for downstream radiomics and biomarker prediction.

The challenge was distinct from general organ segmentation due to the target characteristics: pulmonary nodules in the LIDC-IDRI dataset are often extremely small ($< 6\text{mm}$), possessing low contrast against the surrounding parenchyma, and mimicking pulmonary vasculature.

1.2 Model Selection: Why VISTA-3D?

I conducted an extensive evaluation of candidate architectures, including standard nnU-Net, UNETR, and Swin-UNETR. Ultimately, I selected **VISTA-3D (NVIDIA)** as the foundation model. This decision was driven by three engineering requirements:

1. **Pre-trained Weight Initialization:** Unlike general U-Nets that require random initialization, VISTA-3D provides weights pre-trained on massive datasets. For small object segmentation, starting from a "knowledgeable" backbone is mandatory to avoid convergence failure.
2. **SOTA Status:** As a recently released foundation model, it incorporates modern Transformer-based architectural improvements optimized for medical imaging.

3. **Hybrid Prompting Mechanism:** VISTA-3D supports point-based prompting. This allows us to train a model that can operate in two modes: fully automatic segmentation or interactive segmentation, where a clinician provides a "click" (point prompt) to guide the model. This aligns perfectly with the clinical workflow envisioned for the Virtual Biopsy tool.

1.3 Engineering Constraints and Initial Hurdles

Implementing VISTA-3D presented significant systems engineering challenges. The model was not released with a "plug-and-play" fine-tuning repository. Furthermore, at the time of development, available Large Language Models (LLMs) and documentation had a knowledge cutoff pre-dating the release of the MONAI VISTA SDK.

Consequently, I could not rely on existing APIs. I had to reverse-engineer the model loading process. Initial attempts to reconstruct the architecture using generic *SegResNet* blocks failed because the layer definitions did not perfectly match the pre-trained checkpoints. Through deep investigation of the official codebase and analysis of a "Spline Fine-tuning" example found there, I identified the specific model class and definition (*vista3d132*) required to strictly load the state dictionary without key mismatches.

Chapter 2

Data Engineering Pipeline

The quality of the input data proved to be the single biggest determinant of model performance. Our preprocessing pipeline evolved significantly as we identified flaws in our initial approach.

2.1 Phase 1: The Patch-Extraction Approach (Deprecated)

Our initial strategy, shared across the team, relied on a multiprocessing producer-consumer pattern designed to maximize GPU throughput. This pipeline extracted fixed-size patches from the CT scans and saved them as raw NumPy arrays.

While computationally efficient, this approach was methodologically flawed for a Foundation Model workflow. By saving cropped arrays without their spatial metadata, we lost the affine transformation matrices (direction, origin, spacing). This made it impossible to reconstruct the segmentation back into the patient’s physical coordinate system during inference, rendering the results clinically unusable. Also, this approach created 550GB of processed data, while the new one required only 90GB.

2.2 Phase 2: Full Volumetric Preprocessing (Current Standard)

We completely re-engineered the pipeline to adhere to medical imaging best practices. The new workflow focuses on **Volumetric Consistency**, and we are now working with `.nii.gz` files.

2.2.1 Handling Legacy Dependencies (PyLIDC Patch)

A major blocker was the *pyl IDC* library, which is essential for parsing the LIDC-IDRI annotations. The library depends on deprecated NumPy aliases (e.g., *np.int*, *np.float*) that were removed in recent NumPy versions. This caused the annotation extraction to silently fail or crash. I implemented a runtime "monkey patch" at the start of our scripts to inject these missing aliases back into the system, ensuring stable execution without downgrading our environment.

2.2.2 Consensus and Geometry

The LIDC dataset contains annotations from four different radiologists. To create a single Ground Truth, we implemented a 50% consensus algorithm. If at least two radiologists marked a voxel as a nodule, it was included in the mask.

Crucially, the new pipeline calculates and preserves the **Affine Matrix** for every scan. Instead of saving raw arrays, we now save NIfTI files. This ensures that the physical dimensions of the voxel are preserved, allowing the data loader to perform on-the-fly resampling that is geometrically accurate.

2.3 The Spacing Hypothesis: 1.5mm vs. 0.7mm

One of the most significant findings during my experiments concerned voxel spacing.

- **Initial Configuration:** I started with a target spacing of $1.5 \times 1.5 \times 1.5$ mm, mimicking the VISTA pre-training setup.
- **The Failure Mode:** At this resolution, small nodules (e.g., 3mm diameter) occupied only 2 voxels. The model mathematically could not resolve features, leading to extremely low sensitivity.
- **The Solution:** I consulted the literature and LIDC statistical analysis, identifying the mean scan resolution as ≈ 0.68 mm. I adjusted our target spacing to **0.7mm isotropic**. This increased the volumetric data size by a factor of 10 but was necessary to bring small nodules above the Nyquist limit for feature detection.

Chapter 3

Training Infrastructure and Optimizations

3.1 The Batch Size Constraint

A unique architectural characteristic of VISTA-3D is its inability to handle standard batched inputs during training when using prompts. Because the model accepts a variable number of point prompts per image (e.g., Image A has 2 points, Image B has 5 points), the resulting tensors have mismatched dimensions and cannot be stacked into a single batch.

This forced the training loop to operate with a **Batch Size of 1**. However, training Deep Learning models with a batch size of 1 leads to noisy gradients and unstable convergence.

3.2 Gradient Accumulation Implementation

To solve the batch size limitation, I implemented a custom **Gradient Accumulation** mechanism. Instead of updating the model weights after every image, we process N images sequentially, accumulating their gradients in the background. Only after N images (the effective batch size) do we perform an optimizer step.

I encountered and fixed a critical bug in the early versions of this implementation where the Learning Rate scheduler was stepping too frequently, effectively annealing the learning rate to zero before the model had time to learn. The corrected logic ensures the optimizer and scheduler step in sync with the effective batch size, utilizing a Scaler to handle mixed-precision (AMP) stability.

3.3 Validation Latency and Inference Optimization

A significant operational bottleneck emerged when moving to 0.7mm spacing. Because the volume dimensions increased by a factor of ≈ 10 , the sliding window inference mechanism (required to process the huge volumes) resulted in validation times exceeding **1 hour per epoch**.

I attempted to accelerate this by reducing the sliding window overlap from 0.5 to 0.25. However, this optimization caused a noticeable drop in detection performance for nodules located at the window boundaries (checkerboard artifacts). Consequently, I reverted to 0.5 overlap to ensure accuracy, accepting the computational cost. As I increased the patch size (ROI) in later experiments, validation time naturally decreased (fewer windows required to cover the volume), stabilizing at a manageable duration.

3.4 Context Window vs. Resolution

Moving to 0.7mm spacing introduced a new trade-off. At high resolution, the default patch size of 96^3 covered a very small physical volume. This meant the model often saw a "zoomed-in" view of a nodule without seeing the surrounding lung wall or vessels, leading to false positives.

I conducted a progressive scaling experiment, increasing the patch size from 96 to 128, 160, 192, and finally 224. **Result:** Performance plateaued around 192-224. I found that 192^3 provided the optimal balance between providing enough anatomical context to distinguish vessels from nodules, while fitting within the VRAM constraints of our GPUs.

3.5 Loss Function Dynamics

I experimented with several loss functions. Standard **Dice Loss** proved insufficient because the background class (healthy lung) vastly outnumbers the foreground class (nodule). The model could achieve a low loss simply by predicting "background" everywhere.

I switched to **DiceFocal Loss**, which combines the geometric overlap metric of Dice with the class-imbalance handling of Focal Loss. This forced the model to focus on the "hard" examples (the nodules). I found that providing equal weight ($\lambda = 1.0$) to both components yielded the most stable results.

Chapter 4

Experimental Log and Results Analysis

Throughout the project, I tracked the impact of hyperparameter changes. Below is a narrative summary of the key experiments.

4.1 Zero-Shot Baseline Evaluation

Before initiating the fine-tuning process, I evaluated the official VISTA-3D pre-trained weights (without modification) on my prepared validation set to establish a baseline. The model achieved a mean Dice score of **0.13**.

This low score was highly informative. Detailed error analysis revealed a sharp dichotomy: the foundation model performed excellently on large masses (achieving Dice scores > 0.87 on tumors $> 3\text{cm}$) but completely failed to segment small, sub-centimeter nodules, often treating them as background. This validated my hypothesis: while VISTA-3D understands "Lung Tumors" generally, its pre-training distribution likely skewed towards larger, clinically obvious masses. Fine-tuning was therefore not just an optimization, but a necessity to adapt the domain for early-stage nodule detection.

4.2 Phase 1: The Baseline (Dice 0.44 - 0.54)

The initial baseline using the deprecated preprocessing and 1.5mm spacing achieved a Dice score of roughly 0.44. Switching to DiceFocal Loss provided an immediate jump to 0.54, confirming that class imbalance was a primary bottleneck. I attempted a heuristic post-processing step to remove small objects, but this degraded performance to 0.51, likely because it was filtering out actual small nodules that the model had correctly detected.

4.3 Phase 2: The High-Resolution Era (Dice 0.35 - 0.56)

When I switched to the correct 0.7mm spacing, the metrics initially dropped to 0.35. This was counter-intuitive but expected: the problem became "harder" because the search space increased tenfold.

- **Experiment 16-18 (Scaling ROI):** As I increased the patch size from 96 to 160, the Dice score recovered to 0.47. The model began to understand the difference between a vessel cross-section and a nodule.
- **Experiment 20 (Sampling Strategy):** I adjusted the sampling ratio. Instead of showing the model random healthy lung tissue frequently, I forced the data loader to oversample positive nodule regions (Ratio 2:1). This improved the score to 0.49.
- **Experiment 22 (The Peak):** Combining 0.7mm spacing, a large context window of 192^3 , and the optimized sampling strategy, I achieved our best quantitative real result of **0.56 Dice**.

4.4 Analysis of "Model Collapse"

In several recent high-performance runs, I observed a phenomenon of "Loss Landscape Collapse." The training loss would plummet to near zero, while the validation Dice would fluctuate wildly. This suggests the model is overfitting to the specific geometry of the training nodules or finding a trivial solution to minimize the Focal loss term. I am currently mitigating this by introducing Cosine Annealing for the Learning Rate and experimenting with reduced Focal weights.

4.5 Conclusion on Metrics

While a Dice score of 0.56 may appear numerically modest compared to organ segmentation tasks (which often reach more than 0.90), it is a strong result for small nodule detection. Qualitative analysis of the inference masks shows that the model effectively identifies nodules as small as 3-4mm. The Dice metric penalizes small spatial shifts heavily—a single pixel offset on a 5-pixel nodule reduces the Dice score significantly, even if the detection is clinically useful. Therefore, I conclude that the model is performing better than the raw numbers suggest.